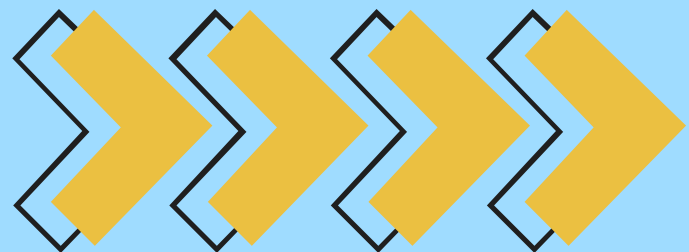


Sistema de Torneio Pokémon

Trabalho 1 – Programação Orientada a Dados

Aline Acevedo



Visão geral:

Sistema que simula um torneio Pokémon a partir da leitura de um arquivo de entrada. O sistema lê os dados, executa as batalhas e gera dois arquivos de saída.

- Torneio/ → pacote com Sistema.py (classes) e Erros.py (erros customizados)
- app.py → aplicação principal que lê o arquivo e executa o torneio
- inputs/ → arquivos de entrada (.txt e typeChart.csv)
- outputs/ → arquivos gerados (Torneio.out e Torneio.err)

Organização do Sistema.py

- **Tipo** → guarda nome e dicionário de efetividade contra outros tipos
- **EfeitoStatus (abstrata)** → Queimadura, Envenenado
- **Golpe** → nome, tipo, poder, acurácia, efeito
- **Pokemon** → atributos calculados, golpes, status
- **Item (abstrata)** → Pocao, SuperPocao, Antidoto, Antiqueimadura, CuraTotal
- **Acao (abstrata)** → AcaoAtq, AcaoItem
- **Treinador** → guarda nome, lista de Pokémon (até 6) e itens; escolhe ação 70/30
- **Batalha** → simula o confronto 1v1 entre dois treinadores, turno a turno
- **Torneio** → gerencia as batalhas, elimina perdedores e guarda o vencedor

Conceitos POO aplicados:

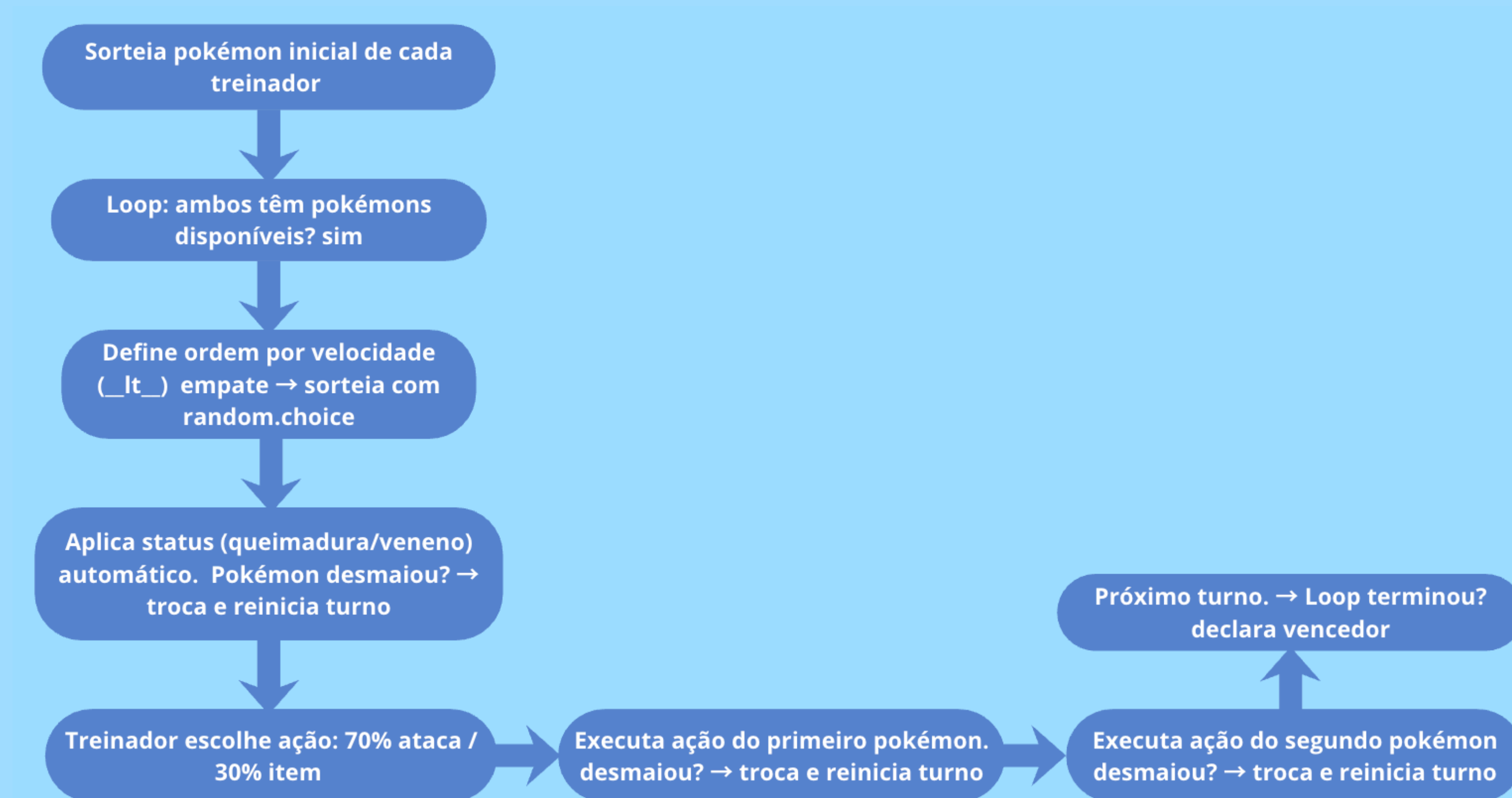
- Herança e classes abstratas
- Encapsulamento
- Sobrecarga de operadores (`__eq__`, `__lt__`, `__str__`)

Erros.py

- ErroNomeMuitoLongo
- ErroPokemonExcedido
- ErroValorInvalido
- ErroElementoDuplicado
- ErroEntradaInvalida



Método `simular()` da classe `Batalha`:



Decisões de implementação:

- **Função clonar() no Pokémon:**

- Sem clonar, treinadores diferentes compartilhariam o mesmo objeto na memória.
- O dano de um afetaria o outro
- Solução: cada treinador recebe sua própria cópia independente

- **Número de turnos no log:**

A cada turno tem uma sinalização no log (Torneio.out)

```
--- Turno 3 ---  
Zapdos usou ThunderShock e causou 7 de dano!  
Reshiram usou DragonPulse e causou 42 de dano!  
--- Turno 4 ---  
Zapdos usou um item!  
Reshiram usou DragonPulse e causou 45 de dano!
```

- **Separação de responsabilidades:**

Leitura e manipulação dos dados no app.py e lógica do "jogo" no Sistema.py

Desafio Solucionado:

- **Problema:**

A partir do Torneio2.txt a execução ficava muito lenta e output suspeitavelmente grande. → demora na execução

- **Tentativa inicial:**

Limitar os turnos em 200 → não era a solução esperada

- **Causa real:**

Quando um pokémon desmaiava, o turno não reiniciava corretamente, o pokémon desmaiado continuava participando da batalha.

- **Solução:**

Corrigir o fluxo do simular() garantindo que, ao desmaiar, o pokémon é trocado e o turno recomeça corretamente.

Pontos para melhorar:

- **Método `simular()` na classe `Batalha` muito extenso:**

Os blocos do primeiro e segundo pokémon são quase idênticos, poderiam virar um método auxiliar como `executar_acao(treinador, pokemon_atacante, pokemon_alvo)`. Deixaria o `simular()` mais legível.

- **Separação do `Sistema.py` em módulos:**

- Poderia separar cada classe (ou grupo relacionado) em seu próprio arquivo dentro do pacote: `Pokemon.py`, `Golpe.py`, `Item.py`, `Tipo.py`, etc.
- Facilitaria a navegação, manutenção e leitura do código
- Deixei porque estava funcionando e não quis arriscar quebrar algo mexendo na estrutura

- **Erro de limite de Pokémon não disparado:**

- O limite de 6 Pokémon é garantido por slicing no construtor do `Treinador`, mas de forma silenciosa
- Defini o `ErroPokemonExcedido` pra registrar esse caso no log, mas não cheguei a levantá-lo

